

Backend — Semana 1: Setup, conexión a la base y autenticación

Objetivo de la semana

Dejar el proyecto de backend andando, conectado a la base de datos provista, con el módulo de autenticación (JWT) funcionando de punta a punta.

Qué se les provee

Script de base de datos ``clinica_ampliada.sql`` (tablas: ``sede``, ``usuario``, ``cobertura``, ``especialidad``, ``medico_especialidad``, ``agenda``, ``turno``, ``historial_clinico``, ``log_auditoria``, ``notificacion``). Para esta semana solo van a necesitar ``usuario`` y ``sede``.

Consigna

Inicialización del proyecto

- Proyecto Node.js con Express.
- Estructura de carpetas: ``src/controllers``, ``src/routes``, ``src/database`` (o similar).
- Variables de entorno (``env``) para credenciales de la base de datos y para el secreto de JWT.

Conexión a la base de datos

- Conexión a MySQL/MariaDB usando el script provisto.
- Probar la conexión con un endpoint simple (ej. ``GET /health`` o similar) antes de avanzar.

Registro de usuario (paciente)

- Endpoint ``POST /auth/registro``.
- Datos requeridos: nombre, apellido, DNI, email, contraseña, fecha de nacimiento y cobertura (``id_cobertura``, elegida de las coberturas existentes en la base, para ello es necesario realizar un servicio que recupere las coberturas disponibles).
- La contraseña **debe** guardarse hasheada con bcrypt — no se acepta texto plano.

- Al registrarse por este endpoint, el usuario queda con rol `paciente` automáticamente.

- Validar que el DNI y el email no estén duplicados.

Login

- Endpoint `POST /auth/login`.

- Recibe dni y contraseña, valida contra el hash almacenado.

- Si es correcto, devuelve un JWT con al menos: `id`, `rol` y `id\_sede` (si aplica) del usuario.

Middlewares de seguridad

- `verificarToken`: valida que el JWT sea válido y no esté vencido; si no, responde 401.

- `verificarRol(...rolesPermitidos)`: valida que el rol del usuario autenticado esté entre los permitidos para ese endpoint; si no, responde 403.

- Armar al menos un endpoint de prueba protegido por cada uno de estos middlewares (por ejemplo, un `GET /auth/perfil` que devuelva los datos del usuario logueado a partir del token).

Formato de respuesta uniforme

- Desde esta primera semana, todos los endpoints deben responder con la estructura uniforme definida en el enunciado general: un **código numérico**, un **texto de estado** (tipo `"ok"` o mensaje de error) y una **propiedad de datos** cuando el servicio devuelva información.

- Esta convención debe aplicarse por igual a las respuestas de éxito y de error (registro, login, endpoints protegidos), y mantenerse consistente en todas las semanas siguientes.

Entregable de la semana

Repositorio con el proyecto backend inicializado.

Endpoints `POST /auth/registro`, `POST /auth/login`, `GET /auth/perfil` (protegido) funcionando.

Colección de Postman (o similar) con las pruebas de estos tres endpoints.

Criterios de aceptación

El proyecto levanta con `npm run dev` (o script equivalente) sin errores.

Las contraseñas se almacenan hasheadas en la base.

El login devuelve un JWT válido, verificable con el secreto configurado.

Un endpoint protegido rechaza (401) una petición sin token o con token inválido.

Un endpoint protegido por rol rechaza (403) a un usuario con rol no permitido.

Todas las respuestas (éxito y error) siguen la estructura uniforme: código numérico, texto de estado y propiedad de datos.